

Examen final - Proyectos Aplicados a la I.E , Sección N

Victor Daniel Tepáz Morente, 202202119, Alejandra Magali López Miranda 201600085

Resumen—Se desarrolló una plataforma web para una clínica veterinaria utilizando el framework Django con Python y una base de datos PostgreSQL. El sistema permite la gestión integral de servicios veterinarios como consultas, vacunaciones, castraciones, lavado y agendamiento de citas en línea. La plataforma está diseñada para ofrecer una experiencia segura y eficiente tanto para los clientes como para el personal administrativo, incorporando módulos de autenticación, validación de datos y una interfaz intuitiva para la solicitud y confirmación de servicios.

I. INTRODUCCIÓN

Este proyecto desarrolla una plataforma web integral para una clínica veterinaria utilizando Django, Python y PostgreSQL, con el objetivo de digitalizar y optimizar la gestión de servicios médicos veterinarios. La solución implementada permite el registro seguro de usuarios, agendamiento de citas, gestión de mascotas y administración de servicios como consultas, vacunaciones y esterilizaciones, incorporando validaciones de datos, confirmaciones vía correo electrónico y una interfaz intuitiva que garantiza una experiencia de usuario fluida y eficiente, representando una modernización significativa en la interacción entre la clínica y sus clientes.

II. OBJETIVOS

II-A. General

Desarrollar una plataforma web robusta y funcional para una clínica veterinaria que optimice la gestión de servicios y mejore la interacción entre clientes y personal, mediante el uso de tecnologías modernas como Django, Python y PostgreSQL.

II-B. Específicos

- * Implementar un sistema seguro de registro y autenticación de usuarios.
- * Integrar una base de datos relacional que garantice la consistencia y disponibilidad de la información de usuarios, mascotas, servicios y citas.
- * Facilitar la administración centralizada de citas y servicios a través de una interfaz accesible y responsive.

III. INVESTIGACIÓN DE UN PROCESO REAL

Como punto de partida para el desarrollo del sistema, se llevó a cabo una investigación orientada a comprender las necesidades reales relacionadas con la gestión de servicios veterinarios en entornos donde la digitalización es limitada o inexistente. La motivación surgió a partir de la observación directa de un proceso frecuente en diversas comunidades:

la ausencia de un mecanismo eficiente para agendar citas veterinarias. En la localidad donde se realizó el estudio, el procedimiento habitual para recibir atención médica para una mascota consiste en presentarse físicamente en la clínica, esperar en fila y depender de la disponibilidad inmediata del personal veterinario. Este método genera incertidumbre, largos tiempos de espera y una saturación del servicio, provocando molestias tanto para los propietarios de mascotas como para el personal de atención.

Durante la visita realizada como parte de la investigación, se documentó que el establecimiento veterinario local no cuenta con un sistema digital de citas, ni un registro ordenado de servicios, horarios disponibles o historial de pacientes. En muchos casos, la información se gestiona de forma manual o a través de notas informales, lo que incrementa el riesgo de pérdida de datos, duplicidad de información o confusión en los horarios. Además, la falta de una plataforma web impide que los usuarios puedan consultar servicios, precios, disponibilidad o requisitos antes de acudir físicamente a la clínica.

Ante esta problemática, se decidió analizar referencias de clínicas veterinarias que sí implementan herramientas digitales modernas. Una de las principales fuentes de inspiración fue el sitio web de la tienda y clínica “El Arca de Noé”, un establecimiento reconocido por su estructura organizada de servicios. Durante la revisión del sitio, se identificaron características fundamentales para el diseño de un sistema funcional, tales como:

- Un catálogo detallado de servicios veterinarios accesible para el cliente.
- Información clara sobre consultas, vacunaciones, esterilizaciones, baños veterinarios, entre otros.
- Secciones bien estructuradas para preguntas frecuentes y descripción de procedimientos.
- Integración de sistemas de contacto rápido para solicitudes de información.

El análisis de este sitio permitió comprender cómo una clínica moderna organiza la información que presenta al usuario, cuáles son los elementos esenciales que un cliente espera encontrar y qué procesos deben ser automatizados para mejorar la eficiencia operativa. Esta comparación resaltó la brecha entre la realidad observada en la comunidad y las posibilidades de una plataforma web moderna.

Con base en la investigación realizada, se definieron los requisitos funcionales del sistema a desarrollar. Entre ellos se incluyeron: un módulo de autenticación y seguridad de usuarios, gestión de perfiles de mascotas, administración de citas mediante un calendario centralizado, panel administrativo para el personal, registro de historial clínico básico y genera-

ción de comprobantes mediante archivos PDF. Se determinó también la necesidad de incorporar una base de datos robusta que garantizara integridad y disponibilidad de la información, así como una interfaz amigable para facilitar la interacción del usuario con el sistema.

El estudio del proceso real evidenció que la implementación de un sistema web no solo optimiza el tiempo de atención, sino que reduce errores humanos, mejora la satisfacción del usuario y aumenta la capacidad operativa de la clínica. Además, permite establecer un registro histórico valioso para el seguimiento de cada mascota, algo que no es posible mediante procesos manuales tradicionales.

En conclusión, la investigación del proceso real fue un elemento clave para definir la arquitectura, funcionalidades y prioridades del proyecto. El análisis de una problemática existente, combinado con la revisión de un modelo digital ya establecido, permitió diseñar una solución adaptada a necesidades concretas y viable para ser implementada en clínicas veterinarias que aún no cuentan con sistemas de gestión modernos.

IV. ALGORITMO

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERIA
CUADERNILLO DE TRABAJO

CARNE: 201600825 SECCIÓN: P

APellidos: Lopez, Marañón
Nombres: Alejandra, Angeli
CURSO: Programación
CARRERA: Electrónica
CATEDRÁTICO: José Silva
FECHA: 06/10/25

ALGORITMO DEL PROGRAMA

1. Iniciar el sistema y mostrar la página principal
2. Hacer iniciar sesión o registrarse
 - 2.1 Si se selecciona registrarse
 - 2.1.1 Solicitar datos (Nombre, apellido, DPI, teléfono, usuario, correo, contraseña)
 - 2.1.2 Confirmar contraseña
 - 2.1.3 Solicitar datos mascota (nombre, especie, sexo, raza, edad, peso)
 - 2.1.4 Solicitar historial médico
 - 2.1.5 Registrar al usuario y mascota en la base de datos
 - 2.2 Si se selecciona iniciar sesión
 - 2.2.1 Solicitar datos (Usuario, contraseña)
3. Sesión iniciada correctamente
 - 3.1 Mostrar menú (Ver servicios, agendar cita, consulta de historial de citas, cerrar sesión)
 - 3.1.1 Si se selecciona "Ver servicios"
 - 3.1.1.1 Mostrar servicios disponibles
 - 3.1.2 Si se selecciona "agendar cita"
 - 3.1.2.1 Solicitar y seleccionar mascota
 - 3.1.2.2 Solicitar tipo de servicio
 - 3.1.2.3 Seleccionar fecha y hora
 - 3.1.2.4 Guardar cita en base de datos y enviar correo de confirmación al usuario
 - 3.1.3 Si se selecciona "consulta de historial de citas"
 - 3.1.3.1 Mostrar historial de citas con fecha, tipo de servicio y estado
 - 3.1.4 Si se selecciona "cerrar sesión"
 - 3.1.4.1 Finalizar sesión
 - 3.1.4.2 Redirigir a la página principal
 4. Modo ADMINISTRADOR
 - 4.1 Crear, editar o eliminar servicios
 - 4.2 Bloquear o desbloquear usuarios
 - 4.3 Descargar datos
5. Finalizar sistema

Fuente: Elaboración propia, 2025

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERIA
CUADERNILLO DE TRABAJO

CARNE: 202202174 SECCIÓN: M

APellidos: Lopez, Marañón
Nombres: Víctor, Daniel
CURSO: Proyecto Aplicados II
CARRERA: Electrónica
CATEDRÁTICO: Amador Silva
FECHA: 06/10/25

Algoritmo del programa

- 1) Iniciar y mostrar la página principal
Mostrar la información de la clínica, los servicios ofrecidos y el contacto
- 2) Verifica si el usuario posee cuenta.
Si no tiene procede a crearla.
Crear una cuenta.
Solicita datos de las mascotas
Solicita el historial médico
Registrar al usuario y a la mascota
Si tiene cuenta reenvía al inicio de sesión
- 3) Sesión iniciada correctamente
Mostrar menú
 - Ver servicios
 - Agendar citas
 - Consultar el historial
 - Cerrar sesión
- 4) Finaliza el sistema

Fuente: Elaboración propia, 2025

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERIA
CUADERNILLO DE TRABAJO

CARNE: 202202174 SECCIÓN: M

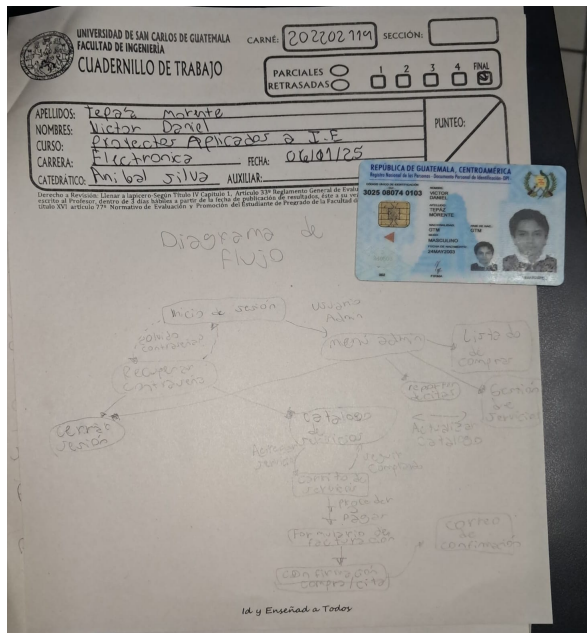
APellidos: Lopez, Marañón
Nombres: Víctor, Daniel
CURSO: Proyecto Aplicados II
CARRERA: Electrónica
CATEDRÁTICO: Amador Silva
FECHA: 06/10/25

Algoritmo del programa

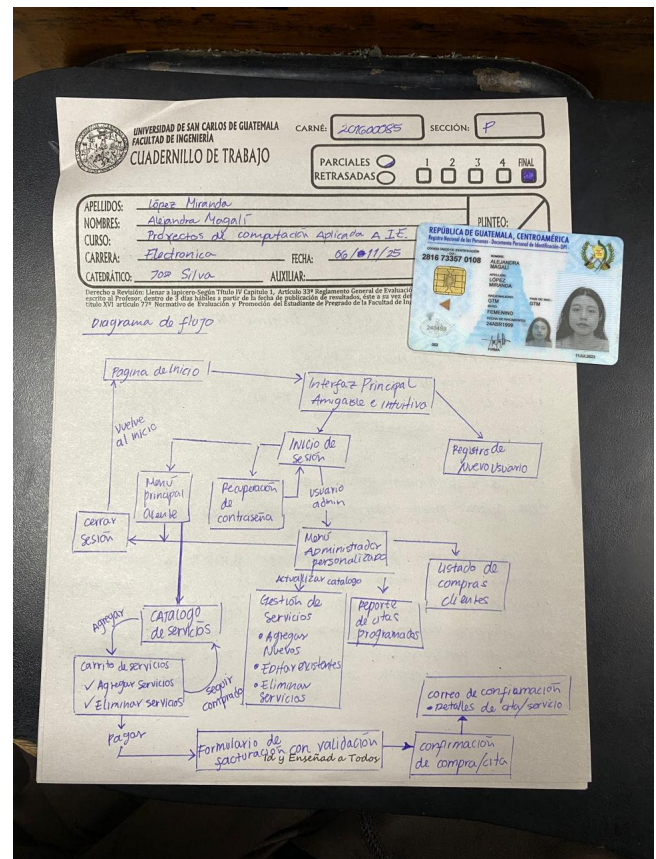
- 1) Iniciar y mostrar la página principal
Mostrar la información de la clínica, los servicios ofrecidos y el contacto
- 2) Verifica si el usuario posee cuenta.
Si no tiene procede a crearla.
Crear una cuenta.
Solicita datos de las mascotas
Solicita el historial médico
Registrar al usuario y a la mascota
Si tiene cuenta reenvía al inicio de sesión
- 3) Sesión iniciada correctamente
Mostrar menú
 - Ver servicios
 - Agendar citas
 - Consultar el historial
 - Cerrar sesión
- 4) Finaliza el sistema

Fuente: Elaboración propia, 2025

V. DIAGRAMA DE FLUJO

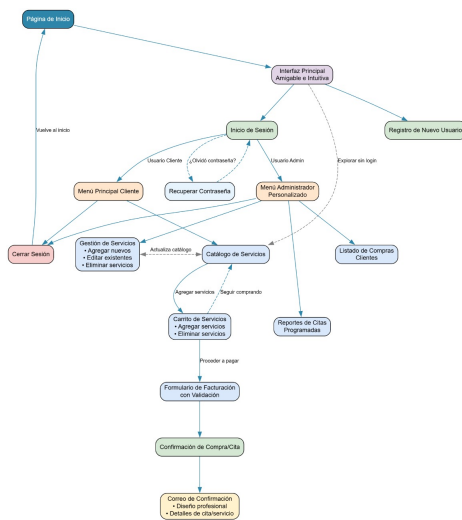


Fuente: Elaboración propia, 2025

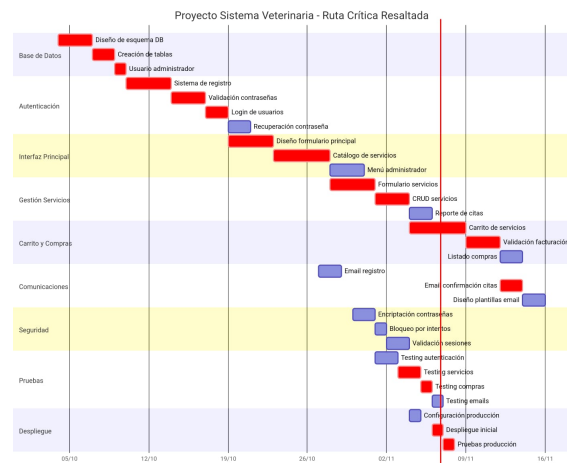


Fuente: Elaboración propia, 2025

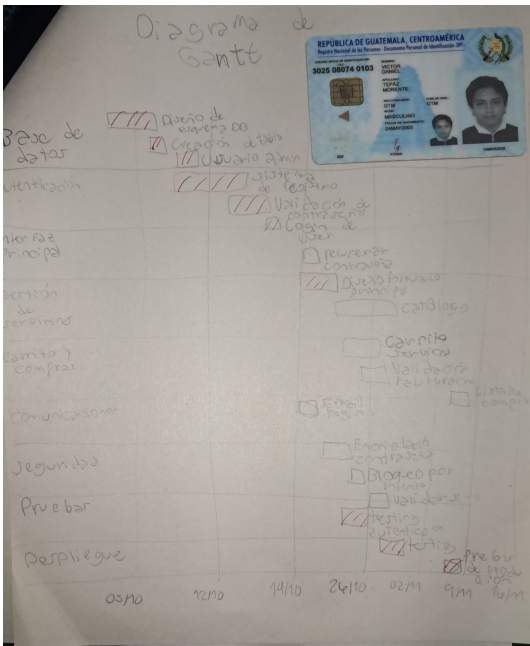
VI. DIAGRAMA DE GANTT



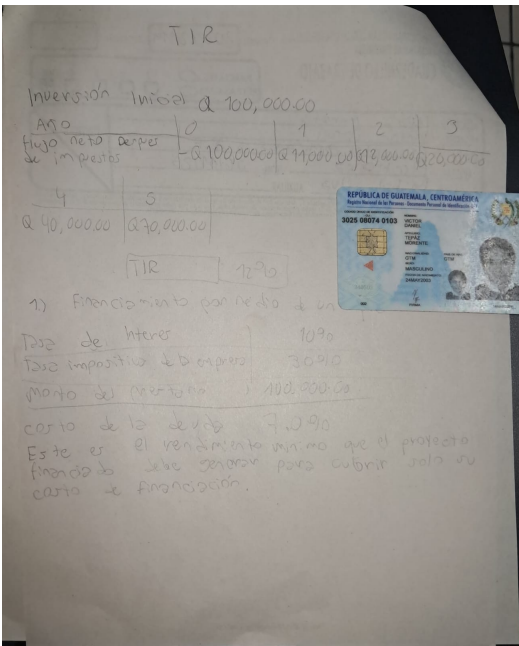
Fuente: Elaboración propia, 2025



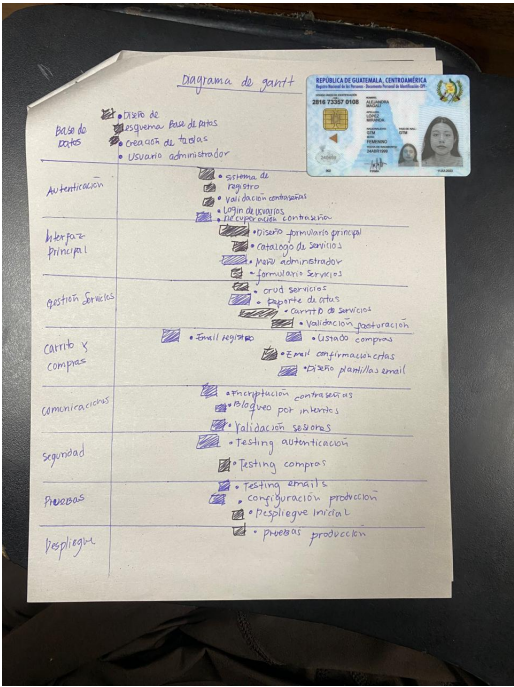
Fuente: Elaboración propia, 2025



Fuente: Elaboración propia, 2025



Fuente: Elaboración propia, 2025

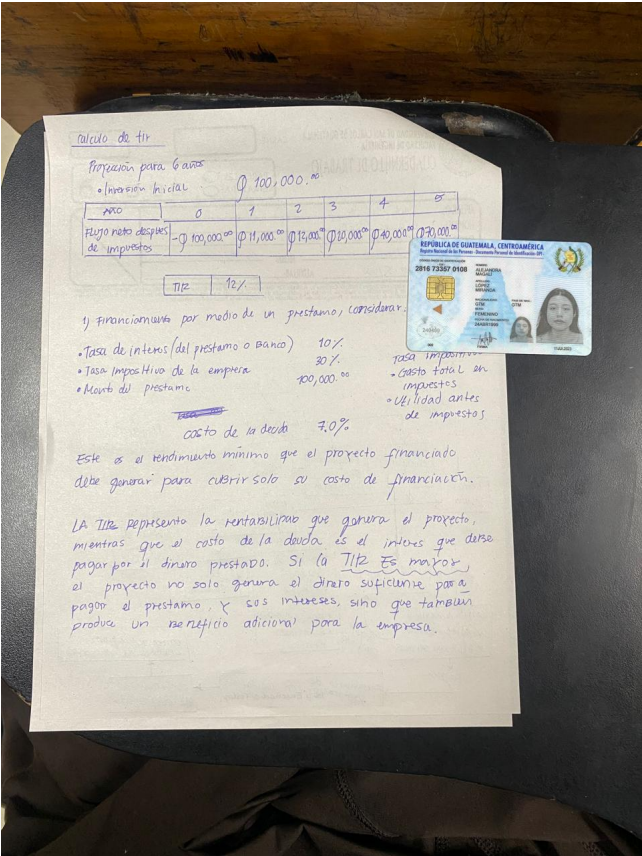


Fuente: Elaboración propia, 2025

VII. TIR

PROYECCION PARA 6 AÑOS										
INVERSION INICIAL	100,000.00									
AÑO	0	1	2	3	4	5				
Flujo neto después de impuestos	Q 100,000.00	Q 11,000.00	Q 12,000.00	Q 20,000.00	Q 40,000.00	Q 70,000.00				
TIR		12%								
1) Financiamiento por medio de un préstamo, considerar:										
Tasa de interés (del préstamo o banco)	10%									
Tasa impositiva de la empresa	30%									
Monto del préstamo	100,000.00									
Costo de la deuda	7.0%									

Fuente: Elaboración propia, 2025



Fuente: Elaboración propia, 2025

VIII. RESULTADOS

VIII-A. Activación del entorno

```
C:\Users\Alejandra\Desktop\980-Proyectos\Proyecto\Django\proyecto\Scripts>activate

(proyecto) C:\Users\Alejandra\Desktop\980-Proyectos\Proyecto\Django\proyecto\Scripts>
```

Fuente: Elaboración propia, 2025

VIII-B. Inicialización del servidor

```
(proyecto) C:\Users\Alejandra\Desktop\980-Proyectos\Proyecto\Django\proyecto>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until
you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
October 06, 2025 - 22:17:01
Django version 5.2.7, using settings 'veterinaria.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Fuente: Elaboración propia, 2025

VIII-C. Estructura de los views utilizado

```
1 from django.contrib import admin
2 from django.urls import path, include
3 from django.conf import settings
4 from django.conf.urls.static import static
5 from usuarios.views import editar_usuario
6 from django.contrib.auth import views as auth_views
7 from usuarios.forms import CustomPasswordResetForm
8 from usuarios.forms import CustomSetPasswordForm
9 from django.contrib import admin
10 from django.urls import path, include
11 from usuarios.views import CustomPasswordResetConfirmView
12 from usuarios.views import CustomPasswordResetConfirmView
13
14 from citas.views import (
15     agendar_cita,
16     historial_citas,
17     editar_cita,
18     eliminar_cita,
19     marcar_completada, # <- aqui
20 )
21
22 from usuarios.views import (
23     inicio, custom_login, registro, redirect_by_role,
24     panel_administracion, perfil_usuario, cerrar_sesion,
25     editar_perfil, eliminar_usuario, detalle_cita,
26     gestion_citas, cambiar_estado_cita
27 )
28 from mascotas.views import (
29     agregar_mascota, editar_mascota, eliminar_mascota
30 )
31 from citas.views import (
32     agendar_cita, historial_citas, editar_cita, eliminar_cita
33 )
34 from servicios.views import (
35     gestion_servicios, crear_servicio, editar_servicio, eliminar_servicio
```

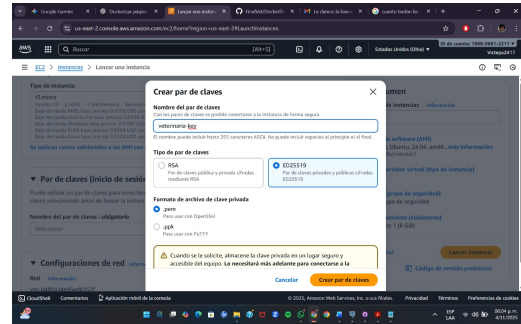
Fuente: Elaboración propia, 2025

VIII-D. Estructura de los modelos utilizado

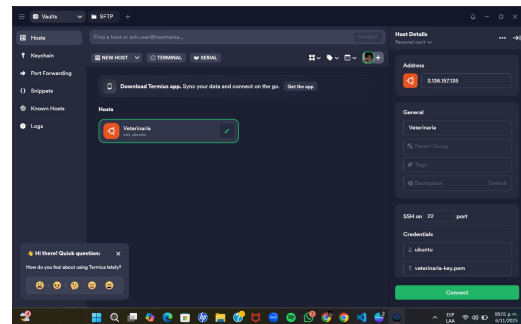
```
1 from django.contrib.auth.models import AbstractUser
2 from django.db import models from django.conf import settings
3 from django.utils import timezone
4
5 class Cliente(AbstractUser):
6     dpi = models.CharField(max_length=13, unique=True)
7     telefono = models.CharField(max_length=15)
8     direccion = models.TextField()
9
10     def __str__(self):
11         return self.username
12
13 class Mascota(models.Model):
14
15 class Servicio(models.Model):
16
17 class Cita(models.Model):
```

Fuente: Elaboración propia, 2025

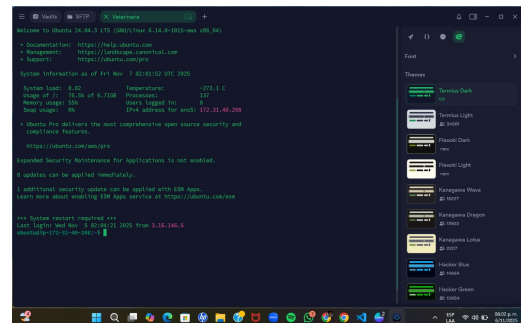
VIII-E. AWS



Fuente: Elaboración propia, 2025



Fuente: Elaboración propia, 2025



Fuente: Elaboración propia, 2025

VIII-F. Pagina corriendo en AWS



Fuente: Elaboración propia, 2025

El desarrollo del sistema web para la clínica veterinaria demostró la eficacia del entorno Django para la construcción de aplicaciones web robustas, modulares y seguras. Django, al ser un framework de alto nivel basado en Python, permitió una rápida estructuración del proyecto gracias a su arquitectura Modelo-Vista-Template (MVT), facilitando así la separación entre la lógica de negocio, la presentación y el acceso a los datos.

Durante el desarrollo, se organizaron diversas aplicaciones dentro del proyecto principal, siguiendo las buenas prácticas que propone Django. Entre estas aplicaciones, se incluyó una dedicada a la gestión de usuarios y autenticación, otra para la administración de servicios veterinarios (como vacunación, consultas, castraciones, etc.), y una más enfocada en la gestión de mascotas y la programación de citas.

Las vistas desarrolladas en Python permitieron manejar la lógica de las peticiones HTTP, procesar formularios y coordinar la interacción entre el usuario y la base de datos. Estas vistas se integraron con formularios de Django para validar datos como el nombre del cliente, la fecha de la cita, el tipo de servicio solicitado y la información de las mascotas. Se utilizaron tanto **vistas basadas en funciones (FBV)** como **vistas basadas en clases (CBV)**, dependiendo de la complejidad de cada caso.

En cuanto a la base de datos, se utilizó **PostgreSQL** como motor relacional por su fiabilidad y compatibilidad con Django. Se estableció una conexión fluida entre la aplicación y la base de datos mediante el **ORM (Object-Relational Mapping)** de Django, lo que facilitó la manipulación de datos sin necesidad de escribir consultas SQL directamente. Se crearon modelos personalizados para almacenar información de los clientes (incluyendo usuario y contraseña encriptada), las mascotas registradas, los tipos de servicios solicitados y las fechas programadas para las citas.

La plataforma resultante permite a los usuarios:

- Registrarse e iniciar sesión de forma segura.
- Registrar información básica de sus mascotas.
- Solicitar servicios según disponibilidad y necesidades.
- Almacenar y consultar su historial de citas.
- Recibir confirmación de citas vía correo electrónico.

IX. CONCLUSIONES

- * El desarrollo con Django demostró ser eficiente para crear una plataforma web robusta y segura, facilitando la estructura modular y el manejo de datos mediante ORM con PostgreSQL.
- * Se implementó exitosamente un sistema integral que permite a los usuarios registrar mascotas, solicitar servicios veterinarios y gestionar citas, con validaciones de seguridad y confirmación vía correo electrónico.
- * La arquitectura MVT y la modularidad del proyecto sentaron las bases para futuras expansiones, demostrando que la solución es escalable y adaptable a nuevas funcionalidades.
- * Docker permitió desplegar el proyecto de forma ordenada y reproducible, encapsulando tanto la aplicación Django como la base de datos PostgreSQL en contenedores

independientes. Gracias a docker-compose, se simplificó el levantamiento y administración de los servicios. En general, Docker facilitó el desarrollo, la portabilidad y la escalabilidad del sistema sin depender de configuraciones locales.

- * AWS brindó la infraestructura necesaria para alojar el proyecto en la nube mediante una instancia EC2. Esto permitió acceder al sistema desde cualquier lugar y mantenerlo funcionando de forma estable. Con la configuración adecuada del servidor y los puertos, AWS demostró ser una plataforma confiable, flexible y segura para el despliegue del sistema veterinario.
- * Termius facilitó la conexión remota al servidor en AWS mediante SSH, haciendo más cómoda y visual la administración de contenedores y archivos. Su uso simplificó el control del entorno de despliegue y agilizó las tareas de mantenimiento del proyecto desde cualquier dispositivo.

X. REPOSITORIO EN GITHUB

El código fuente, ejemplos y materiales utilizados para esta comparación se encuentran disponibles en el siguiente repositorio público de GitHub:

<https://github.com/AlejandraMLM/980-Proyectos/tree/main/veterinaria>

Este repositorio contiene los códigos implementados en Python.

XI. LINK DEL VIDEO

Video con la explicación del proyecto

<https://youtu.be/m8zNbRBvPoY>

XII. ANEXOS

XII-A. Reunion del grupo



Fuente: Elaboración propia, 2025

XIII. BIBLIOGRAFÍA

REFERENCIAS

- [1] MathWorks. (2023). *MATLAB Documentation*. Disponible en: <https://www.mathworks.com/help/matlab/>
- [2] Eaton, J. W., Bateman, D., Hauberg, S., Wehbring, R. (2022). *GNU Octave Manual Version 7*. GNU Project. Disponible en: <https://www.gnu.org/software/octave/doc/interpreter/>

- [3] Van Rossum, G., Drake, F. L. (2023). *The Python Language Reference*. Python Software Foundation. Disponible en: <https://docs.python.org/3/>
- [4] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array programming with NumPy*. *Nature*, 585(7825), 357–362. Disponible en: <https://numpy.org/>
- [5] Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. *Computing in Science & Engineering*, 9(3), 90–95. Disponible en: <https://matplotlib.org/>